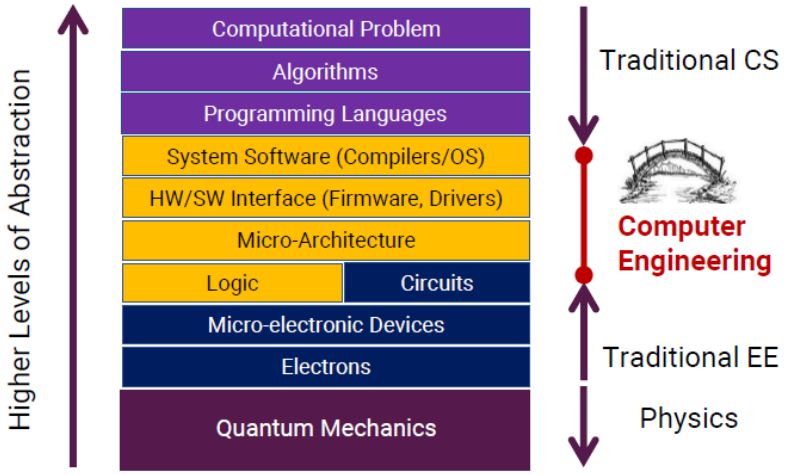
Week1

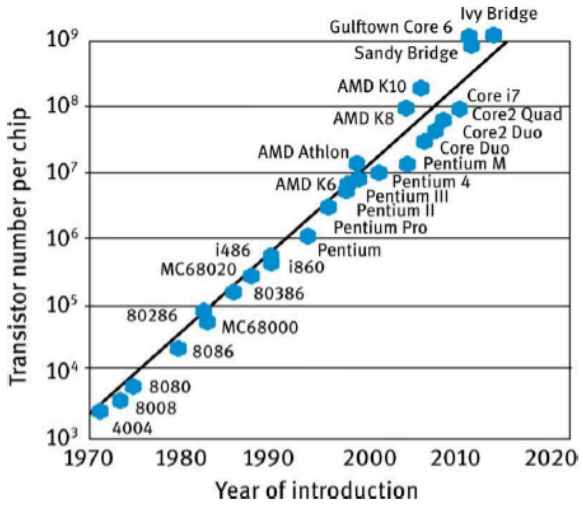
- Computer systems are not restricted to laptop/desktop, but devices like smartwatches, washing machines, robots, drones, smart phones or even a self-driving car.
- Every computer system has the following sub-units: *Computing (processing) unit*, *Communication unit*, *Memory/storage unit*. *Communication unit* acts as the bridge between the *Computing unit* and the *Memory/storage* and lets the computing unit read/write data from/to it. THis design was devised by von Neumann and recorded in his seminal work in 1946.
- Transformation hierarchy.



- At its most fundamental level, a computer solves problems by orchestrating the movement of electrons. Microelectronic devices, such as transistors, facilitate this process and are arranged within circuits. These circuits are designed using logic principles and collectively form the microarchitecture, which serves as the foundation of hardware functionality.
- On top of the microarchitecture, a hardware/software interface acts as a bridge, connecting the underlying hardware components with the software layer that manages the entire system. System software—including compilers and operating systems—configures and controls various hardware elements such as memory and storage units. At the highest level of abstraction, high-level programming languages and algorithm-driven programs enable users to interact with the system and perform complex tasks efficiently.
- Smart phone→PCBs(~8/device)→ICs(8-16/PCB)→Modules(8-10/IC)→Cells(1K-10K/module)→Gates(2-16/cell)→Transistors(2-8/gate)
- VLSI is used to build such devices that pack billions of transistors into it.
- Transistor allows two states - on and off, 1 and 0, up and down, high and low.
- Moore's law is a term used to refer to the observation made by Gordon Moore in 1965

that the number of transistors in a dense integrated circuit (IC) doubles about every two years.

- History of shrinking chips.



First transistor 1947 (Nobel prize)

First Silicon transistor 1954

First CMOS gate 1963

First commercial CMOS chips 1974 (digital watches)

As of 2020, some of the largest chips have >10'000'000'000 transistors, about as many (or more that) people on Earth.

- Storage should be small, inexpensive, stable (reliable, repeatable), easy to store/access/transform/combine/transmit, and enable fast manipulation.
- *Voltage* of charged particles is primarily used to encode information. Advantages with this choice is easy generation of *voltage*, very low power requirement. DIsadvantages include effect of noise, resistive and capacitive effects on slowing down things.
- Use of analog value to store information can lead to ambiguity, less reliability and reproducibility.
- Copy/Invert are pre-packaged functionality available at the hardware level. However, if we repeat copy/invert operations an even number of times in a sequence, they won't be able to precisely reproduce the input voltage due to noise that gets added at each stage.
- In the case of digital representation of data, binary digits/bits (or a string of such entities thereof) are used to store data. Note that a bit has only two *legit/legal* values - 0 and 1. This will allow to effectively "ignore" the noise at each stage and hence able to reproduce the original input precisely, without distortion. In this case, a pre-decided voltage represents each of the two states . Note that these voltage values must be same throughout the entire system. Thus, *0V* could represent 0, whereas *5V* could represent 1.
- Let's assume the minimum possible voltage is 0V and the maximum possible voltage is 5V, establish a threshold voltage V_{TH}. If the input voltage $v < V_{TH}$, then interpret the bit as 0. If the input voltage $v \geqslant V_{TH}$, then interpret the bit as 1. The issue with this system is the effect of noise that could potentially alter the interpreted value. Due to this anomaly, there's an alternative system wherein a *forbidden range* is introduced, in which $v \leqslant V_L$ is interpreted as 0, $v \geqslant V_H$ is interpreted as 1, and $V_L > v > V_H$ is

forbidden.

- A collection of n bits gives 2^n possible values. For example, 3 bits can represent 8 possible values.
- Boolean algebra was invented by George Boole in 1854. Digital Electronics was born in 1940's, thanks to Claude Shannon who noticed the similarities between Boolean algebra and electronic telephone switches.

Week2

- Logic circuits are built using logic gates comprising of transistors.
- In practice, the output may be generated after a certain delay from the time input is supplied.
- Combinational logic is also termed 'memoryless' logic, since the output is dependent only on the current inputs, and not on any of the previous sets of inputs. This is different from the sequential logic, which stores the current inputs and decide the output in the next cycle, apart from the inputs during the next cycle.
- *AND (.)*, *OR (+)*, *NOT* operations are the most basic boolean algebra operations.
- $A\ XNOR\ B$ is represented by the boolean expression $AB + \overline{AB}$.
- $A\ XOR\ B$ is represented by the boolean expression $\overline{A}B + A\overline{B}$.
- Axioms of Boolean Algebra

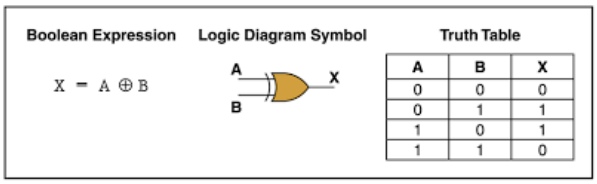
Formal version	English version
1. *B* contains at least two elements, *0* and *1*, such that $0 \neq 1$	**Math formality...**
2. *Closure* $a,b \in B$ (i) $a + b \in B$ (ii) $a \bullet b \in B$	**Result of AND, OR stays in set you start with**
3. *Commutative Laws*: $a,b \in B$, (i) $a + b = b + a$ (ii) $a \bullet b = b \bullet a$	**For primitive AND, OR of 2 inputs, order doesn't matter**
4. *Identities*: $0, 1 \in B$ (i) $a + 0 = a$ (ii) $a \bullet 1 = a$	**There are identity elements for AND, OR, that give you back what you started with**
5. *Distributive Laws*: (i) $a + (b \bullet c) = (a + b) \bullet (a + c)$ (ii) $a \bullet (b + c) = a \bullet b + a \bullet c$	**• distributes over +, just like algebra … but + distributes over •, also**
6. *Complement*: (i) $a + \bar{a} = 1$ (ii) $a \bullet \bar{a} = 0$	**There is a complement element; AND/ORing with it gives the identity elm.**

- Variables and literals are not the same. In the boolean expression $A + \overline{A}B$, there are two variables A and B, but 3 literals since A and $\overline{A}$ are considered as two different

literals.

- Dual of a boolean expression is derived by replacing:
  - All *AND* operations with *OR*
  - All *OR* operations with *AND*
  - Every *1* with *0*, and every *0* with *1*.
  - None of the complements should be changed.
- For example, dual of the expression $X.(Y+Z)$ is $X+(Y.Z)$
- All axioms remain true in its dual form.
- Idempotent law states $X+X=X$. Its dual is $X.X=X$
- Involution law is $\overline{(\overline{X})}=X$. In this case, there is no dual, since complement operation cannot be changed.
- Laws of complementarity states $X+\overline{X}=1$. Its dual is $X.\overline{X}=0$
- Association law states that $(X+Y)+Z=X+(Y+Z)=X+Y+Z$. Its dual is $(X.Y).Z=X.(Y.Z)=X.Y.Z$
- Some simplification theorems
  - $X.Y+X.\overline{Y}=X.(Y+\overline{Y})=X.1=X$
  - $X+X.Y=X.(1+Y)=X.1=X$
  - $(X+\overline{Y}).Y=X.Y+Y.\overline{Y}=X.Y+0=X.Y$
  - $X+\overline{X}.Y=X+(\overline{X}.Y)=(X+\overline{X}).(X+Y)=1.(X+Y)=X+Y$
- DeMorgan's law:
  - $\overline{X+Y+Z+...}=\overline{X}.\overline{Y}.\overline{Z}....$
  - $\overline{X.Y.Z....}=\overline{X}+\overline{Y}+\overline{Z}+...$
- Product of some (or all) of the input variables is called *implicant*. Product of all input variables is called *minterm.* For example, the row 0-0-1-1 in the truth table of a boolean expression with 4 input variables *a, b, c, d* can be represented by the *minterm* $\overline{a}\,\overline{b}cd$.
- Sum of all input variables is called *maxterm*. For example, the row 0-0-1-1 in the truth table of a boolean expression with 4 input variables *a, b, c, d* can be represented by the *maxterm* $\overline{a}+\overline{b}+c+d$.
- According to *operator precedence, NOT* has the highest precendence, followed by *AND,* and finally *OR.* Thus, $A.\overline{B}+C.D$ is evaluated as $(A.(\overline{B}))+(C.D)$
- If multiple gate realizations give rise to same truth table, it's perfectly valid to choose any of them. Choose the simplest, the most efficient, one with the most common gates (say *NAND* gates), the one with the least number of layers.
- Steps to simplify a boolean expression into a *Sum of Products(SOP)* expresssion:
  - Pick rows from the truth table with an output of 1. Construct the *minterms* for each row.
  - *OR* all the minterms.

- XOR (exclusive OR) gate is represented using the symbol $\oplus$. Thus, if A and B are the inputs, $A \oplus B$ represents the exclusive OR operation of its inputs. It's equivalent to $\overline{A}B + A\overline{B}$ or $(A + B)(\overline{A} + \overline{B})$. It's graphically represented as



- XNOR is the complement of the XOR. It's equivalent to $AB + \overline{A}\overline{B}$ or $(\overline{A} + B)(A + \overline{B})$
- $A.B$ can be implemented using 2 NAND gates as $\overline{\overline{A.B}}$. It can be implemented using 3 NOR gates as $\overline{\overline{(A + A)} + \overline{(B + B)}}$
- $A + B$ can be implemented using 3 NAND gates as $\overline{\overline{A.A}.\overline{B.B}}$. It can be implemented using 2 NOR gates as $\overline{\overline{(A + B)}}$
- https://www.youtube.com/watch?v=w2hK0JVKmJc

Week3

- In a standard shorthand notation for SOP (called canonical form), list rows in a truth table in a natural "numeric" ordering $0,\ 1,\ 2,\ 3...$. Here $m4$ implies the rows where the input variables take values $A = 1,\ B = 0,\ C = 0$.
- For example, given that there are 3 input variables, $\Sigma m(3, 4, 5, 6, 7)$ implies the following truth table.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

  Note that the number of input variables must be known before hand to derive the truth table.
- Note that canonical form is NOT necessarily minimal.
- To compute the result of $A.B.C.D$, you could either use
  - 3 AND gates in sequence wherein $A.B$ is computed at level-1, fed to level-2 AND gate, combine with C to compute $A.B.C$, which is then fed to level-3 AND gate, combine with D to finally compute $A.B.C.D$.
  - 2 AND gates that would parallely compute $A.B$ and $C.D$, before combining them at level-2 to get $A.B.C.D$. In this case too, 3 AND gates are used for the

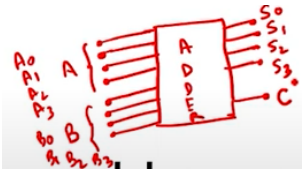
final circuit realization.

Note that in the first case, it uses 3 levels of gates and hence could cause more delay than in the second case, which uses only 2 levels of gates.

- While dealing with circuit implementations, complementing of variables isn't considered as a separate level.
- Product of Sums (POS) is the other canonical form used to represent boolean expressions, by picking the rows from the truth table that result in 0. Thus, the above truth table can be represented by $(A+B+C)(A+B+\overline{C})(A+\overline{B}+C)$ in POS form. Alternatively, it can be written in shorthand as $\Pi M(0,1,2)$
- *POS* of F is the same as the DeMorgan of *SOP* of $\overline{F}$. Thus, for the above truth table, $F(A,B,C) = \Sigma m(3,4,5,6,7) \rightarrow \overline{F}(A,B,C) = \Sigma m(0,1,2)$. In POS representation of the same truth table, take DeMorgan of $\Sigma m(0,1,2)$, which will result in $\Pi M(0,1,2)$.
- *NAND* gates are more commonly available in the market, since it's easier to manufacture and inherently more power efficient. For this reason, NAND (similarly, NOR) gate is called a universal gate.
- SOP expressions can be implemented using two levels of *NAND* gates. For example, $AB+CD = \overline{(\overline{AB+CD})} = \overline{(\overline{AB}.\overline{CD})}$
- Similarly, POS expressions can be implemented using two levels of NOR gates. For example, $(A+B).(C+D) = \overline{\overline{(A+B).(C+D)}} = \overline{\overline{(A+B)}+\overline{(C+D)}}$
- Timing diagrams can be sliced up into pieces, in order to derive the truth table. Once the truth table is available, create the boolean expression, simplify the expression with the help of axioms and finally implement the circuit.
- [implementing boolean expression using minimum number of NAND gates]

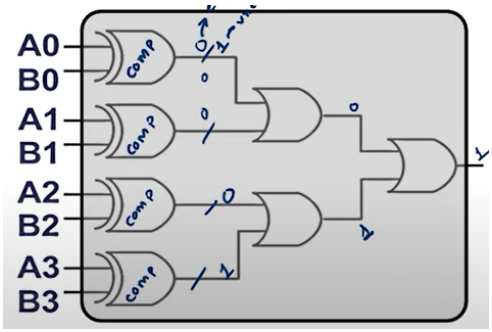
Week4

- Combinational logic is often grouped into larger building blocks to build more complex systems. For example, here's the represention of an adder block that adds two 4-bit numbers, outputs a 4-bit number, along with a carry bit.



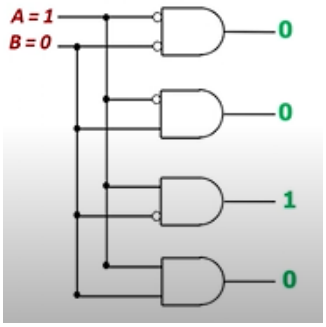
- Comparator circuit could take two inputs x and y, and output a bit to indicate if the given inputs are related as follows: $x \geqslant y$ or $x < y$. Alternatively, related as follows: $x == y$ or $x! = y$
- Programmable Logic Arrays (PLAs) are hardware circuits that could be 'programmed' to 'act' as one of the above mentioned circuits.

- *XOR* and *XNOR* gates are used to realize 1-bit equality comparator circuits. Hence, these gates can be also be used in/as error-checking circuits.
- Here's the circuit that can be used to compare multi-bit (4-bit) binary numbers.

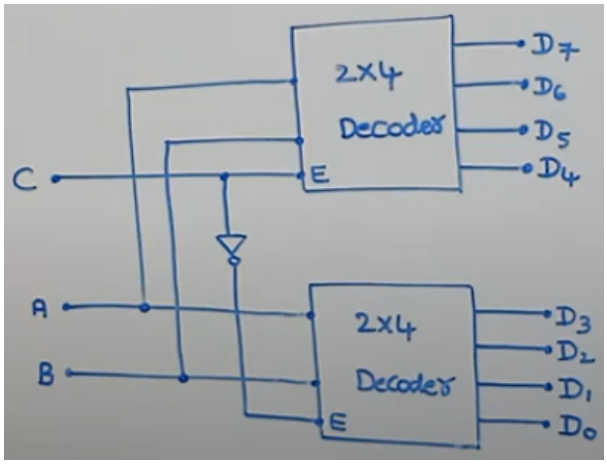


If any of the pairs (A0, B0), (A1, B1), (A3, B3), (A4, B4) are unequal, the output of the above circuit will be 1.

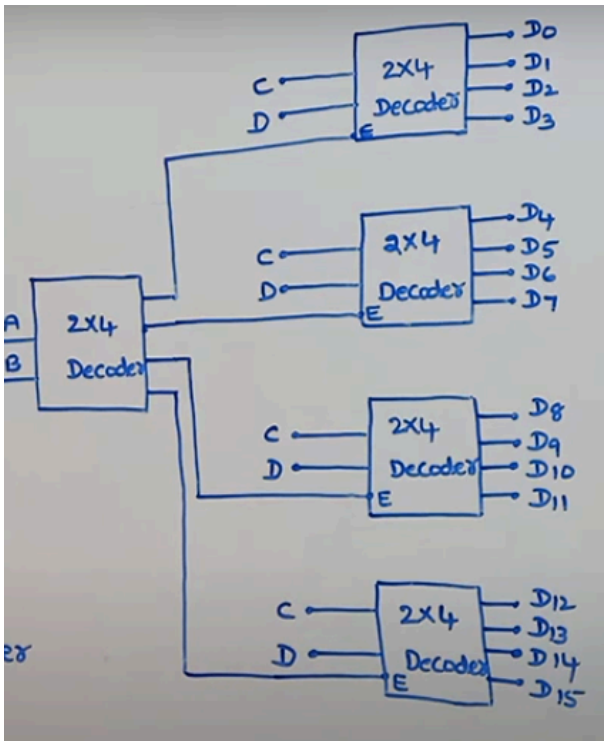
- *XOR* also serves as a half-adder (only "sum" bit) circuit for single-bit numbers. *AND* gate can be used for simulating the "carry" bit.
- Decoder circuit takes n inputs and activates one of the 2^n lines. This effectively converts binary to decimal value. For example, the below circuit takes two inputs 1 and 0, and activates the third output line.



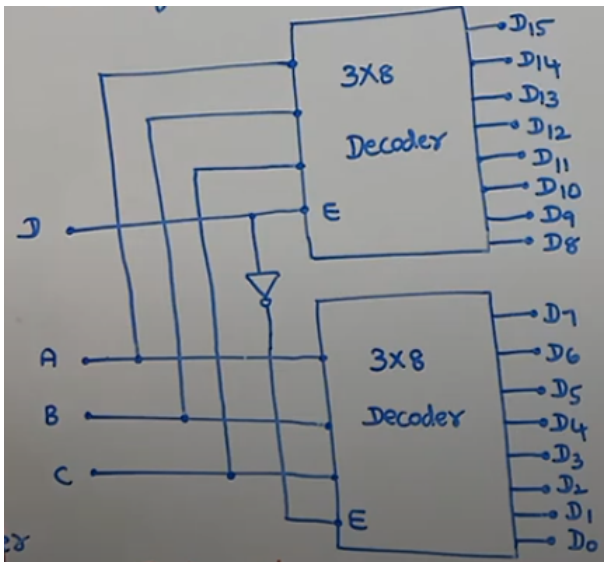
- Decoder could be used to select an address location (0-based) from the DRAM, or for the CPU to decide what action to do based on the instruction opcode.
- Decoder circuit optionally uses *enable* line to activate/deactivate the circuit.
- Decoder block could be used to construct more complicated *functions* by choosing the appropriate minterms.
- 3:8 decoder using 2:4 decoders



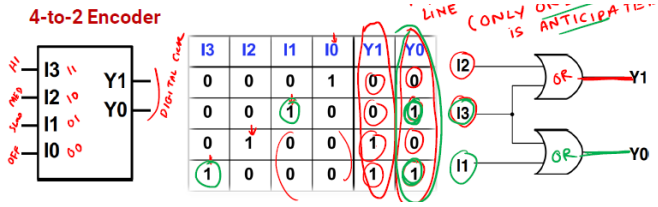
- 4:16 decoder using 2:4 decoders.



- 4:16 decoder using 3:8 decoder



- To find the number of 2-to-4 line decoders required to construct a 7-to-128 line decoder, we repeatedly divide by 4, which is $\frac{128}{4} + \frac{32}{4} + \frac{8}{4} = 32 + 8 + 2 = 42.$
- Encoder circuit does the exact opposite of a decoder. It takes 2^n (decimal) inputs and encodes to a binary.



I3	I2	I1	I0	Y1	Y0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

In the above case, $Y_0 = I_1 + I_3$. $Y_1 = I_2 + I_3$

- In a *priority* encoder circuit, typically only one input is 1. However, when multiple inputs are asserted, highest numbered inputs (starting with MSB) have priority. For example, consider the following truth table.

I3	I2	I1	I0	Y1	Y0	None
1	X	X	X	1	1	0
0	1	X	X	1	0	0
0	0	1	X	0	1	0
0	0	0	1	0	0	0
0	0	0	0	0	0	1

MORE VIDEOS

Here any one among the 8 inputs *1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111* would yield the same output *Y1=1, Y0=1*. As far the MSB is *1*, the other bits doesn't matter and hence termed *don't care* bits. Note that *don't care* bits can essentially be ignored when deriving the minterms, in order to generate the final output.

In the above case, $Y_1 = I_3 + \overline{I_3} I_2 = I_3 + I_2$. $Y_0 = I_3 + \overline{I_3}\,\overline{I_2} I_1 = I_3 + \overline{I_2} I_2$.

- Multiplexers are circuits that helps select any one of the inputs based on a *selection criteria.* Selection is provided by $log_2 N$ bit control input. THus, to control 2 inputs (2:1 MUX), the select line must have 1 bit ($log_2 2 = 1$).
- Here's the truth table (and its alternative representation) of a 2:1 MUX.

S	A0	A1	Y
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

or its alternative

S	Y
0	A0
1	A1

In order to create this circuit, derive the SOP expression from the corresponding truth table and proceed as usual. In this case, the expression is $\overline{S} A_0 + S A_1$.

- Here is the truth table of a 4:1 MUX.

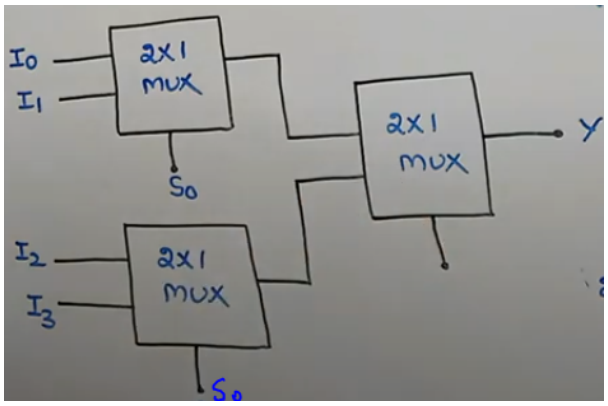
S0	S1	A0	A1	A2	A3	Y
0	0	0	x	x	x	0
0	0	1	x	x	x	1
0	1	x	0	x	x	0
0	1	x	1	x	x	1
1	0	x	x	0	x	0
1	0	x	x	1	x	1
1	1	x	x	x	0	0
1	1	x	x	x	1	1

or its alternative

S0	S1	Y
0	0	A0
0	1	A1
1	0	A2
1	1	A3

This is equivalent to $\overline{S_0}\,\overline{S_1} A_0 + \overline{S_0} S_1 A_1 + S_0 \overline{S_1} A_2 + S_0 S_1 A_3$.

- 4:1 MUX using 2:1 MUX

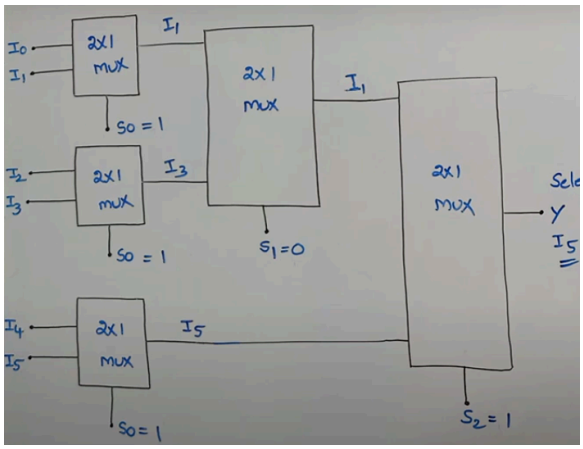


Note that there are two levels in the arrangement, with each level adding one new selection line into the mix.

- In order to construct an 8:1 MUX using 2:1 MUX, there will be 3 levels in the final arrangement, with the level-1 having four 2:1 MUXes, level-2 having two 2:1 MUXes, and level-3 having one 2:1 MUX. At all MUXes in level-1, S_0 is used as the selection line; at all MUXes in level-2, S_1 is used as the selection line and at all MUXes in the

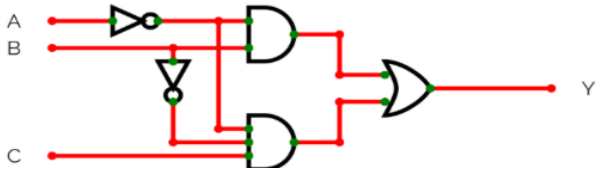
level-3, S_2 is used as the selection line.

- To construct 128:1 MUX using 2:1 MUXes, we need to create multiple levels of 2:1 MUX arrangements. Note, level-1 will have 64 2:1 MUX laid out in parallel, outputs of which serve as inputs to 32 2:1 MUX at level-2. This continues until there's a single output. THus, the total number of 2:1 MUXes needed to accomplish this is given by $\frac{128}{2}+\frac{64}{2}+\frac{32}{2}+\frac{16}{2}+\frac{8}{2}+\frac{4}{2}+\frac{2}{2}=64+32+16+8+4+2+1=127$. Thus, you'll need 127 2:1 MUXes. At each level, you'll introduce a new selection line, and hence 7 selection lines will be required in the entire arrangement.
- 6:1 MUX using 2:1 MUX

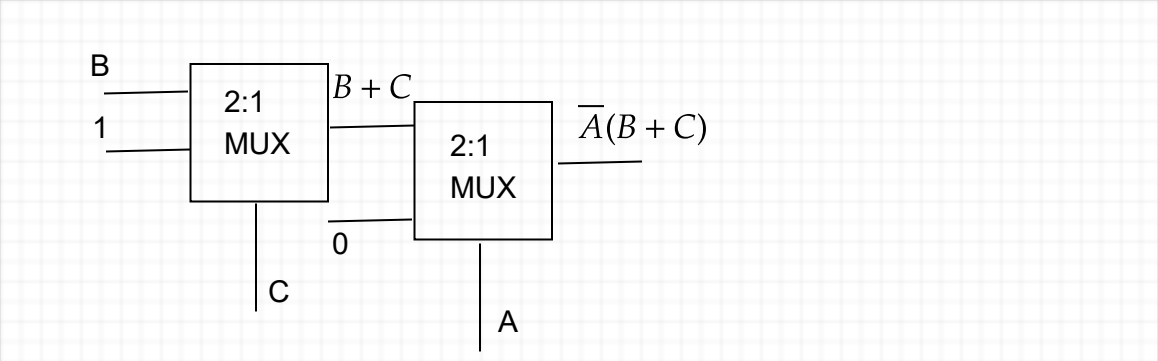


- https://www.youtube.com/watch?v=Uakx03u2o3Q (Past term live session for Week-4)
- Refer to this Youtube playlist for discussion on MUX.
- Refer to this Youtube playlist for discussion on decoders and encoders.
- Problems

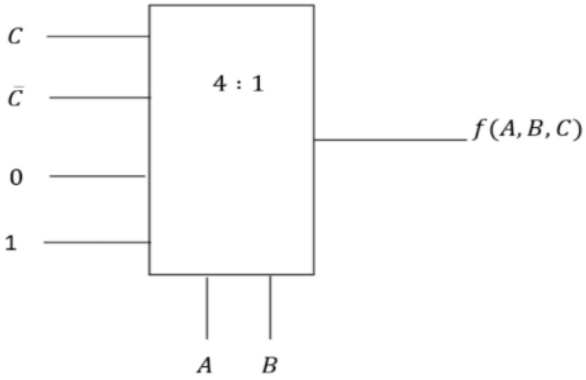
  1. How many 2:1 MUXes are needed to implement the following circuit, and how?



In the above circuit, $Y=\overline{A}B+\overline{A}\overline{B}C$. This can be simplified to $\overline{A}(B+C)$. This can be implemented as



2.



The canonical sum of product (SOP) expression f(A, B, C) is .

Here's the truth table of the above MUX configuration. Note that the minterms are highlighted in grey.

A	B	C	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

THe equivalent SOP is $\overline{A}\overline{B}C + \overline{A}B\overline{C} + AB\overline{C} + ABC$, which can also be represented as $\Sigma(1, 2, 6, 7)$. Alternatively, represented as $\Pi(0, 3, 4, 5)$

3.

A RAM chip has a capacity of 1024 words of 8 bits each (1K × 8). How many 2 × 4 decoders (with enable line) is needed to construct a 16K × 16 RAM from 1K × 8 RAM

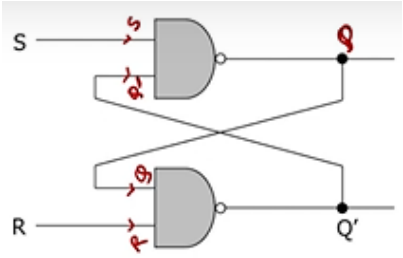
https://gateoverflow.in/1555/gate-cse-2013-question-46?show=101517#a101517

Week5

- To construct a 1-bit full adder, use XOR and AND gate for the sum and carry bits respectively.
- To construct a 4-bit full adder, use 4 1-bit adders in sequence (cascade/ripple). Note that $S = A_i \oplus B_i \oplus C_i$ and $C_{i+1} = A_iB_i + B_iC_i + A_iC_i$. Note that i indicates the place.
- The downside to this cascade/ripple design is that final adder circuit will have a sum total of the delay caused by each of the 1-bit full adders - since each bit must wait for the carry from the previous stage. Thus, a 32 bit adder would result in a total delay of 32δns, if we assume each one-bit adder contributes δns. *Carry-lookahead* adders are a solution to this and reduces the delay.
- PLA is a common building block for implementing any collection of logic functions. It has built-in *AND* gates and *OR* gates as two different layers. *AND* gates simulate every possible minterm, and *OR* gates help combine them. HDL (Verilog, VHDL) helps select necessary *AND* gates out of the available minterms and feed them in the *OR* gates, to simulate any function.
- While creating K-maps (Karnaugh maps), it's important to flip only one variable in each step, when dealing with multiple variables. Thus, 00-01-11-10 is the order, instead of 00-01-10-11 (which is the convention). This numbering scheme is called G*ray Code*.
- Here is the process
  - Circle all the rectangular blocks of 1's in the map, using the fewest possible number of circles. Each circle should be as large as possible. There should be 2^k cells in a group, where k is an integer. Thus, circle can contain 1, 2, 4, 8 etc cells.
  - If there're *don't care* bits in the truth table, use it to create larger circles by suitably assuming the cell to be 1 or 0.
  - Determine which literals are constant, and which vary across group.
  - Eliminate varying literals, then *AND* the constant literals (Use X if 1, and $\overline{X}$ if 0).
  - *OR* together all such terms created from individual groups.
- K-map adjacenicies can go around the edges. Wrap around the first to last column, and top to bottom row.

Week7

- SR latch has the following configuration where two NAND gates are cross-coupled. When $S = 0 \; and \; R = 1, \; Q = 1$ and when $S = 1 \; and \; R = 0 \; , \; Q = 0$. When $S = 1 \; and \; R = 1$, this configuration serves as a memory element, wherein $Q = Q_{prev}$. Avoid the state where $S = 0, \; R = 0$. In this state, $Q = 1 \; and \; \overline{Q} = 1$. This violates the basic assumption that the outputs of the SR latch has to be complements of each other.



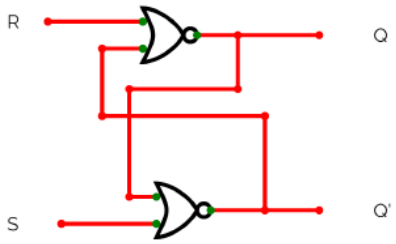
- Thus, the truth table of the above circuit is as follows:

Input		Output
R	S	Q
1	1	Q_{prev}
1	0	1
0	1	0
0	0	Forbidden

Looking into the circuit again,

$Q_{next} = \overline{(S.\overline{Q})} = \overline{(S.\overline{Q.R})} = \overline{S.(\overline{Q}+\overline{R})} = \overline{S} + \overline{(\overline{Q}+\overline{R})} = \overline{S} + QR$

- The condition where $S = 0\ and\ R = 0$ is also called the metastable condition, because if the S and R transition back to 1 at the sametime, the outputs will depend on which NAND gate is faster. If the top NAND gate is faster, $Q = 1\ and\ \overline{Q} = 0$. If the bottom NAND gate is faster, $\overline{Q} = 1\ and\ Q = 0$.
- Here is the NOR configuration of the SR latch.



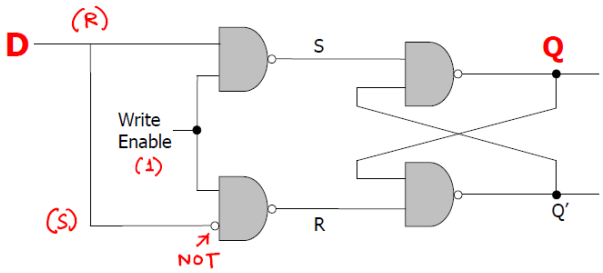
Truth table of the above circuit is as follows

S	R	Q
0	0	Q_{prev}
0	1	0
1	0	1
1	1	Forbidden

Note that the forbidden and memory states have switched, in this case.

- In a gated D-latch, the input S and R are generated by two NAND gates that accept a

write enable (WE) signal and single input D. In effect, Q follows D.

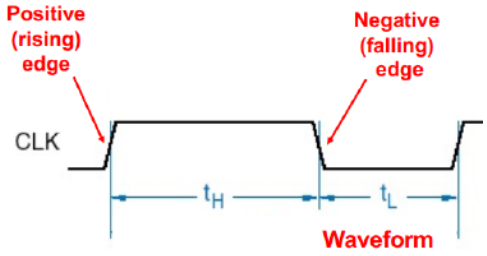


Note that, the above circuit guarantees that S and R are never 0 at the sametime. This makes it impossible for it to enter into the forbidden state. However, they can be 1 at the sametime, if the *write enable (WE)* signal is set to 0, thus acting as a memory element.

- The truth table of the above circuit is as follows:

Input		Output
WE	D	Q
0	0	Q_{prev}
0	1	Q_{prev}
1	0	0
1	1	1

- Clock is an input (typically WE) to a sequential circuit that changes output and state values at a predetermined rate. By default, we use the positive rising edge as the trigger edge for the gate (called one clock tick).

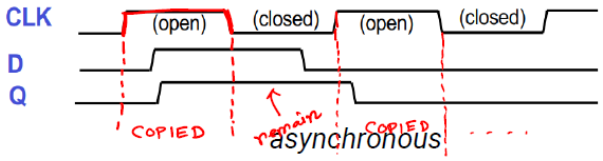


$t_H + t_L$ represents the time spent in one clock cycle. If n such cycles occur in one second, the circuit is said to work at a clock frequency of $\frac{1}{n} Hertz$.

- $\frac{t_H}{t_H + t_L}$ is called duty cycle and equals 50% when, $t_H = t_L$.
- The circuit might use either of the following trigger mechanisms.
  - Edge trigger mechanism. In this case, writing into the gate outputs occurs only at the rising/falling clock edges.
  - Level trigger mechanism. In this case, writing into the gate outputs occur when

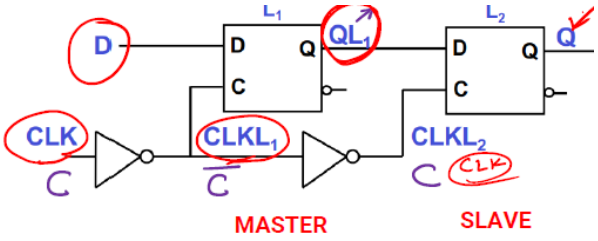
the clock level is 1.

- D-latch is a level-sensitive instrument, which will copy D to Q, when the clock is high. When the clock is low, Q retains its value.



Note that this leads to asynchronous behaviour.

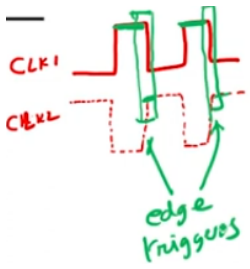
- On the other hand, flipflop is an edge-sensitive instrument, which will copy D to Q only on the clock edge. Otherwise (when the clock is steady at 0 or 1), Q retains its value.
- D flipflop is constructed using 2 back-to-back D-latches connected as below.



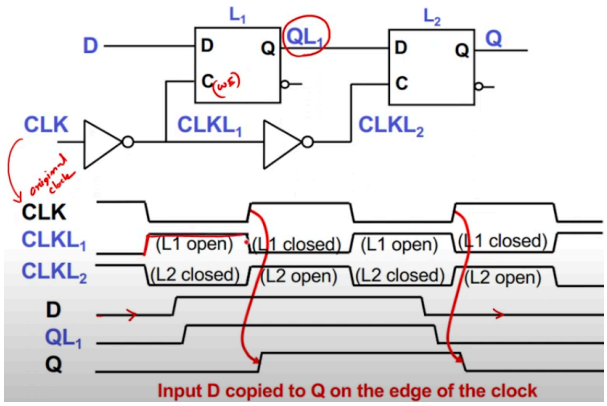
This results in the following truth table.

D	CLK	Q_{next}
0	↑	0
1	↑	1
X	0	(Last) Q
X	1	(Last) Q

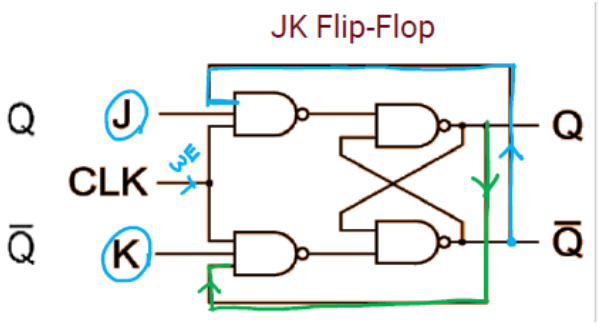
This truth table can be understood using the following diagram of the clocks $CLKL_1$ and $CLKL_2$. Considering $CLKL_1$ as the clock for the first D latch, it is enabled only when $CLKL_1$ is high. Similarly, considering $CLKL_2$ as the clock for the second D latch, it is enabled only when $CLKL_2$ is high. When both clocks are considered together, the final output Q will follow the input D, only during the rising clock edge of original clock CLK.



- The following is the timing diagram of a D flipflop. Note that Q changes follows D only at the rising edges of CLK.

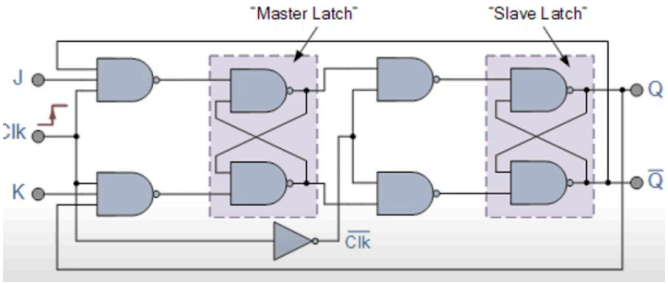


- JK flipflop (drawn below) is an advancement over D flipflop in that there are two separate inputs J and K, rather than a single input D.



When $J = 0\ and\ K = 0$, the flipflop goes into the memory/hold mode. When $J = 0\ and\ K = 1$, the flipflop goes into the reset mode. When $J = 1\ and\ K = 0$, the flipflop goes into the set mode. When $J = 1\ and\ K = 1$, the flipflop goes into a toggling mode, wherein the current value of Q toggles continously and results in an uncontrolled oscillations, as long as the clock input is HIGH. Technically, the above device is a latch (since it's level-triggered).

- In order to avoid fix this, there are two methods.
  - Timing pulse must be very short - switch to low at a speed higher than that of the NAND gate layers. However, this is not practically very feasible.
  - Use a master-slave configuration. This results in an edge-triggered device, and works even when the clock pulse is not very short.



- Note that in the above configuration, feedback occur from the slave to the flipflop

inputs. There is no feedback from the master to its inputs.

- Here's the truth table of the above circuit

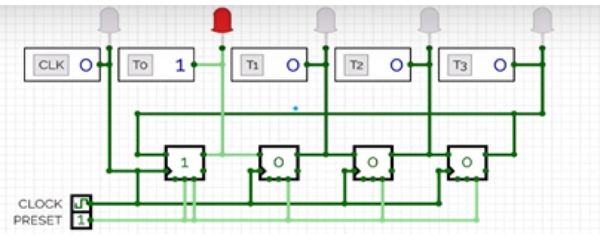
Inputs			Outputs		
J	K	CLK	Q	$\bar{Q}$	Comments
0	0	↑	Q_0	$\bar{Q}_0$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	$\bar{Q}_0$	Q_0	Toggle

- Note that in most literature JK flipflop implies a master-slave configuration that has two *JK latches* connected in a sequence.
- While designing circuits using such devices, *excitation tables* are very useful. Excitation table shows what should be the J/K inputs in order to change the *present* state to the *next* state. Here is the excitation table for a master-slave configured JK flipflop.

Q Output		Inputs	
Present State	Next State	J_n	K_n
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

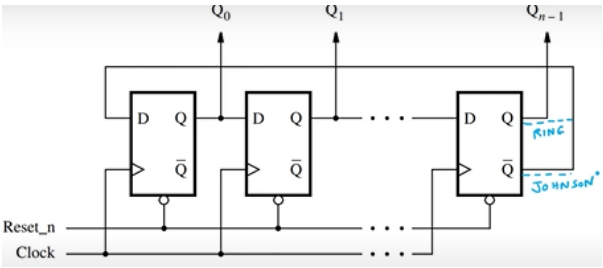
Week9

- In a ring counter, only one bit is 1 at any point in time; in other words, a one-hot encoded state. Thus the sequence, $100,\ 010,\ 001,\ 100$ is generated by a 3-bit ring counter. Note that each state represents $divide - by - 2$ of the previous state.
- An n-bit ring counter generates a sequence of length n.
- in Johnson counter, only one bit is *flipped* at any point in time. Thus the sequence, $000, 100,\ 110,\ 111,\ 011,\ 001,\ 000$ represents a Johnson counter. Note that the sequence thus generated is called a gray-code sequence.
- An n-bit Johnson counter generates a sequence of length $2n$.
- Following circuit shows a 4-bit ring counter, where 1 rotates through the flip-flops, upon each rising edge for the next 4 clock cycles. Note that the D1 is fed with the Q3 (output of D4), thus making the rotation continuous.



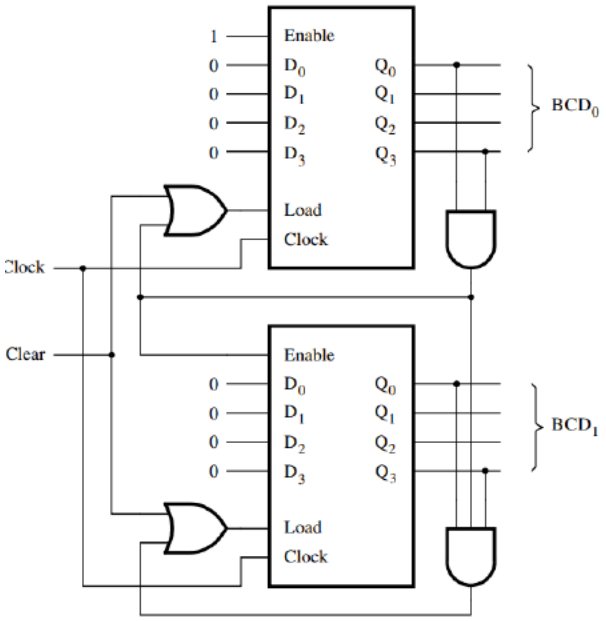
There is an important part that makes this design work - the pulse $PRESET$ and $ASYNC\ RESET$. Both are both set to 1 for all flip-flops during the start of operation of the counter. This ensures that $D1$ to be initialized with 1. $ASYNC\ RESET$ ensures that all flip-flops other than $D1$ gets 0. As soon as the counter starts its operation, $PRESET$ and $ASYNC\ RESET$ must be set to 0 (manually) for the flip-flops to start their the normal operation and hence rotates the 1 over the subsequent rising clock edges. This causes 1 to move in a *ring* fashion from left to right, and restart from the left.

- The following circuit shows a Johnson counter, where $\overline{Q}$ of the last flipflop is fedback to the first flipflop at its input. Due to this, the circuit doesn't require a $PRESET$. However, all flipflops need $ASYNC_RESET$ for the normal operation of the counter.

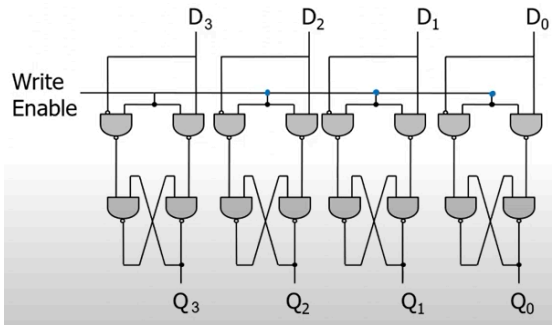


Note that Q (instead of $\overline{Q}$) is fedback in the case of a ring counter.

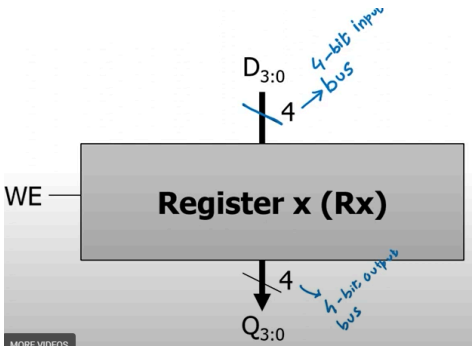
- In a 2 digit BCD counter, there are 2 counters, each counting from $0000\ (0)\ to\ 1001\ (9)$. Note that each counter uses 4 flipflops. When a digit is $1001\ (9)$, the next significant (decimal) digit will be incremented by connecting it to its $ENABLE$ pin. Also, it initializes itself and starts counting from 0000 via its $LOAD$ pin. Further, when the total count is 99, the second counter also gets initialized to 0000 via its $LOAD$ pin.



- Registers are a structures that stores more than one bit and can be read from or written to, depending on the $WRITE\ ENABLE$. Shown below is a register created using D-latches. It can be hold 4 bits of data, referenced as $Q[3:0]$



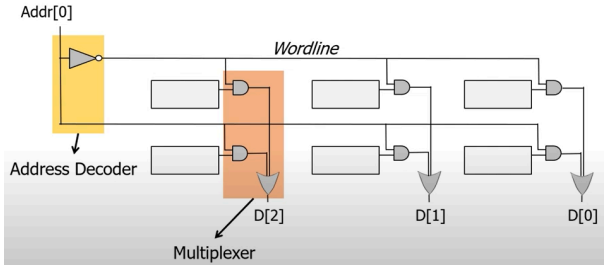
This can be symbolically represented using the following diagram.



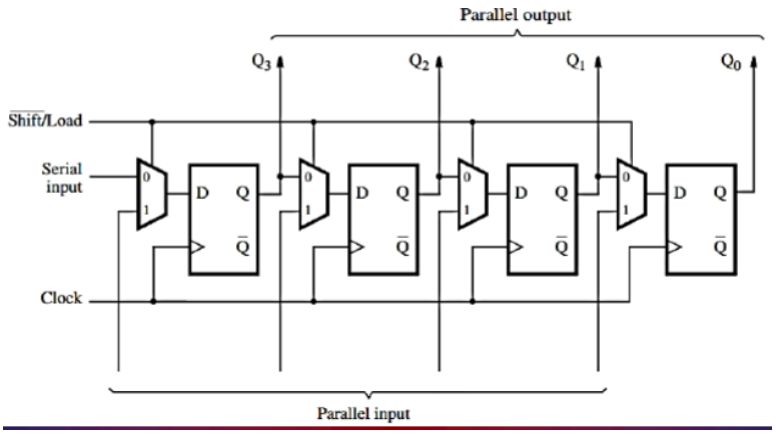
- Memory Device is a collection of such registers. Each slot of memory uses a register and has a unique address. Note that it's only possible to address an entire slot, and

nothing less than it. Thus, if the addressability of memory is 1 byte, its only possible to read a full byte (8 bits) out of it, and not the individual bits, or not even 4 bits.

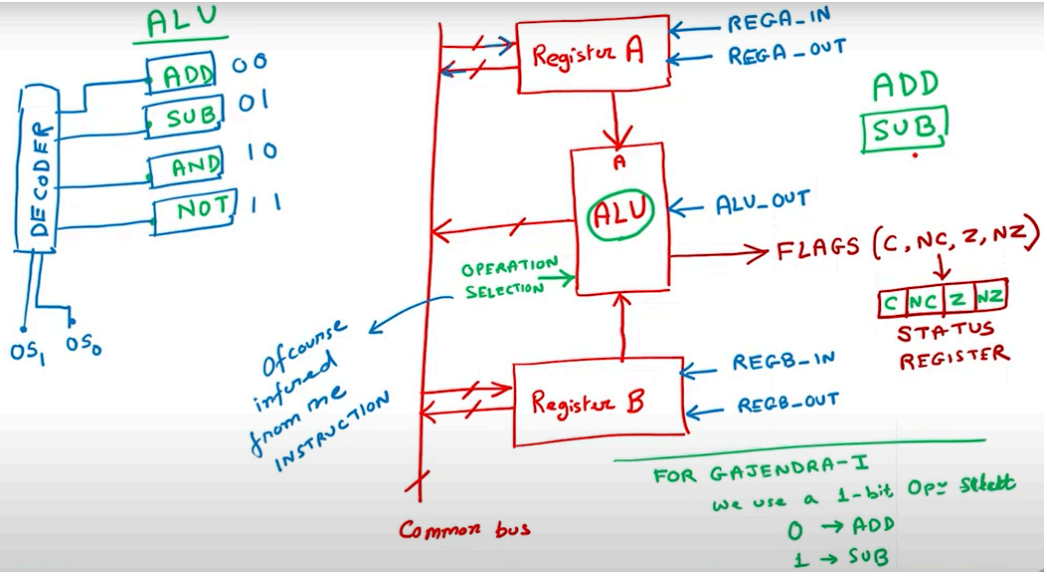
- With a 10 bit address, one can *address* an address space of 1024 (2^{10}) locations. In this case, if the addressability was 4-byte, the total memory the space can hold is $1024 * 4 = 4KB$.
- Address *decoder* decides which memory slot has to be activated. The output bits come from different memory slots and uses a *multiplixer*. The following diagram shows how this works with a 3-bit addressable memory with 2 slots. Note that each of the top/bottom slots can be addressed using a 1-bit decoder. Each slot contains 3 bits.



- Shift registers are devices that shift a bit among the flipflops (in either direction), upon edge trigger. It works much like the ring counter, except for the fact that there is no feedback from the last flipflop the first.
- One of the applications of shift register is a serial communication from sensor to driver and vice-versa. The communication channel is referred to as a *serial bus,* and typically uses *SPI* protocol. Bit-width of the bus is also called the *window*. In this case, for n-bit input data to be copied to the receiver, it takes n clock pulses.
- Shown below is a circuit that accept serial/parallel data, using a multiplixer connected to each D-flipflop. If Shift/Load is 0, it uses serial input and shifts data serially in 4 clock edges. If the Shift/Load is 1, it uses the parallel inputs and outputs to Q[3:0] in a single clock edge.



•



- Control words generated by $LDA \ < addr >$ are as follows.

PC	PC	PC	MEM	MEM	IR	IR	MAR	RA	RA	RB	RB	RC	RC	ALU
INC	O	L	IN	O	IN	O	IN	IN	O	IN	O	IN	O	O
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	1	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0	0	0	0	0	0

Note that here, each row in the above table represents one micro-instruction. During each clock cycle, one micro-instruction gets executed. There are 4 clock cycles used by the above instruction $LDA \ < addr >$, each of which is summarized below.

$$CLK1: PC_OUT, MAR_IN$$
$$CLK2: PC_INC, MEM_OUT, IR_IN$$
$$CLK3: IR_OUT, MAR_IN$$
$$CLK4: MEM_OUT, REG_A$$

In the $CLK3$ above, IR_OUT is activated, and the MAR_IN reads the 4 least significant bits. Though the instruction has 16 bits, MAR is hard-wired only to read the 4 least significant bits ($< addr >$)

- *ADD* and *SUB* instructions has the same set of micro-instructions (control words), picturized below.

PC	PC	PC	MEM	MEM	IR	IR	MAR	RA	RA	RB	RB	RC	RC	ALU
INC	O	L	IN	O	IN	O	IN	IN	O	IN	O	IN	O	O
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	1	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	1

Note that there are 5 micro-instructions to execute $ADD \ < addr >$.
$SUB \ < addr >$ also takes the same micro-instructions. This is because, after it fetches the instruction, it decodes $< addr >$ passed in the instruction, loads it to the REG_B, followed which the ALU perform ADD (or SUB) operation and outputs its contents to the REG_A. The only difference between ADD and SUB operations is in the way *ALU* treats its inputs.